



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

**Faculty of
Computing**

SECP3623

Database Programming

2025/2026 SEMESTER 1

Assignment 1 (10%)

Name	:	EVELYN GOH YUAN QI
Matric Id	:	A23CS0222
Section	:	1

Name	:	LIM YU HAN
Matric Id	:	A23CS0241
Section	:	1

MARKING RUBRICS

Task	Criteria	Marks	Total Marks
Task 1 Database Creation & Constraints	Creation of database and all tables with valid keys, datatypes, and constraints (PK, FK, NOT NULL, UNIQUE). Logical order and referential integrity preserved.	3	
	ALTER TABLE (add email UNIQUE) and DROP TABLE demonstration executed successfully and documented.	2	
	Insertion order, valid FK references, and no orphan data.	2	
Task 2 Data Manipulation & Filtering	Five tables populated with ≥10 valid records each; varied and meaningful data UPDATE/DELETE executed correctly with logical results.	1	
	Accurate use of WHERE, BETWEEN, LIKE, IN, AND/OR/NOT queries.	3	
	Correct use of aggregate and string functions (COUNT, CONCAT, and more) .	2	
Task 3 Reporting & Aggregation	VIEW creation	2	
	Use of GROUP BY, HAVING, ROUND, and CASE with accurate aggregated results.	3	
	Clear labelling, readable outputs, screenshots show executed results.	2	
Total (20)			

Task 1: Database Creation and Constraints

1. Create database

- 1 • `DROP DATABASE IF EXISTS hostel_mgmt_Evelyn;`
- 2 • `CREATE DATABASE hostel_mgmt_Evelyn;`
- 3 • `USE hostel_mgmt_Evelyn;`

Action Output				
	#	Time	Action	Message
✓	1	11:29:59	DROP DATABASE IF EXISTS hostel_mgmt_Evelyn	0 row(s) affected
✓	2	11:29:59	CREATE DATABASE hostel_mgmt_Evelyn	1 row(s) affected
✓	3	11:29:59	USE hostel_mgmt_Evelyn	0 row(s) affected

2. Create table room_types

```
8 • CREATE TABLE room_types (  
9     type_id INT AUTO_INCREMENT PRIMARY KEY,  
10    type_name VARCHAR(50) NOT NULL UNIQUE,  
11    rent DECIMAL(10,2) NOT NULL,  
12    deposit DECIMAL(10,2) NOT NULL,  
13    capacity INT NOT NULL CHECK (capacity > 0)  
14 );
```

✓ 11 11:38:06 CREATE TABLE room_types (type_id INT AUTO_INCREMENT PRIMARY KEY, type_name VARCHAR... 0 row(s) affected

3. Create table rooms

```
13 • CREATE TABLE rooms (  
14     room_id INT AUTO_INCREMENT PRIMARY KEY,  
15     type_id INT NOT NULL,  
16     room_no VARCHAR(10) NOT NULL UNIQUE,  
17     floor_no INT NOT NULL,  
18     is_occupied TINYINT(1) NOT NULL DEFAULT 0,  
19     FOREIGN KEY (type_id) REFERENCES room_types(type_id)  
20 );
```

⚠ 21 11:48:43 CREATE TABLE rooms (room_id INT AUTO_INCREMENT PRIMARY KEY, type_id INT NOT NULL, ro... 0 row(s) affected, 1 warning(s): 1681 Integer display width is deprecated and will be removed in a future r...

4. Create table students

```
26 • CREATE TABLE students (  
27     student_id INT AUTO_INCREMENT PRIMARY KEY,  
28     room_id INT,  
29     fname VARCHAR(50) NOT NULL,  
30     lname VARCHAR(50) NOT NULL,  
31     status ENUM('ACTIVE', 'NON_ACTIVE') NOT NULL DEFAULT 'ACTIVE',  
32     checkin_date DATE,  
33     email VARCHAR(100),  
34     FOREIGN KEY (room_id) REFERENCES rooms(room_id)  
35 );
```

✓ 27 11:55:16 CREATE TABLE students (student_id INT AUTO_INCREMENT PRIMARY KEY, room_id INT, fname V... 0 row(s) affected

5. Create table maintenance

```
32 • CREATE TABLE maintenance (  
33     maint_id INT AUTO_INCREMENT PRIMARY KEY,  
34     room_id INT NOT NULL,  
35     issue_desc VARCHAR(255) NOT NULL,  
36     severity ENUM('LOW', 'MEDIUM', 'HIGH') NOT NULL,  
37     status ENUM('OPEN', 'RESOLVED') NOT NULL DEFAULT 'OPEN',  
38     reported_on DATE NOT NULL,  
39     resolved_on DATE DEFAULT NULL,  
40     FOREIGN KEY (room_id) REFERENCES rooms(room_id)  
41 );
```

✓ 28 11:55:16 CREATE TABLE maintenance (maint_id INT AUTO_INCREMENT PRIMARY KEY, room_id INT NOT N... 0 row(s) affected

6. Create table payments

```
51 • CREATE TABLE payments (  
52     payment_id INT AUTO_INCREMENT PRIMARY KEY,  
53     student_id INT NOT NULL,  
54     amount DECIMAL(10,2) NOT NULL CHECK (amount >= 0),  
55     paid_on DATE NOT NULL,  
56     method ENUM('CASH','FPX','CARD','TNG') NOT NULL,  
57     note VARCHAR(255),  
58     FOREIGN KEY (student_id) REFERENCES students(student_id)  
59 );
```

✓ 29 11:55:17 CREATE TABLE payments (payment_id INT AUTO_INCREMENT PRIMARY KEY, student_id INT NOT... 0 row(s) affected

7. Alter table

```
60     -- ALTER TABLE  
61 • ALTER TABLE students  
62     ADD CONSTRAINT uq_students_email UNIQUE (email);
```

✓ 47 13:21:52 ALTER TABLE students ADD CONSTRAINT uq_students_email UNIQUE (email) 0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0

8. Drop table demonstration

```
64     -- DROP TABLE demonstration  
65 • CREATE TABLE IF NOT EXISTS temp_test_table (  
66     id INT AUTO_INCREMENT PRIMARY KEY,  
67     info VARCHAR(100)  
68 );  
69 • INSERT INTO temp_test_table (info) VALUES ('test1'), ('test2');  
70     -- Now drop it  
71 • DROP TABLE IF EXISTS temp_test_table;
```

✓ 48 13:21:52 CREATE TABLE IF NOT EXISTS temp_test_table (id INT AUTO_INCREMENT PRIMARY KEY, info VA... 0 row(s) affected

✓ 49 13:21:52 INSERT INTO temp_test_table (info) VALUES (test1), (test2) 2 row(s) affected Records: 2 Duplicates: 0 Warnings: 0

✓ 50 13:21:52 DROP TABLE IF EXISTS temp_test_table 0 row(s) affected

Task 2: Data Manipulation and Filtering

1. Insert at least 10 room types

```
76 • INSERT INTO room_types (type_name, rent, deposit, capacity) VALUES  
77     ('Single', 350.00, 200.00, 1),  
78     ('Double', 500.00, 300.00, 2),  
79     ('Premium', 900.00, 500.00, 2),  
80     ('Family', 1200.00, 700.00, 4),  
81     ('Single-Deluxe', 450.00, 250.00, 1),  
82     ('Double-Deluxe', 600.00, 350.00, 2),  
83     ('Premium-Plus', 1000.00, 600.00, 2),  
84     ('Family-Suite', 1500.00, 800.00, 5),  
85     ('Economy', 380.00, 150.00, 1),  
86     ('Executive', 2000.00, 1000.00, 3);
```

✓ 76 13:43:21 INSERT INTO room_types (type_name, rent, deposit, capacity) VALUES ('Single', 350.00, 200.00, 1), (D... 10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0

2. Insert at least 10 rooms

```
90 • INSERT INTO rooms (type_id, room_no, floor_no) VALUES
91 ((SELECT type_id FROM room_types WHERE type_name='Single'), 'A101', 1),
92 ((SELECT type_id FROM room_types WHERE type_name='Double'), 'A102', 1),
93 ((SELECT type_id FROM room_types WHERE type_name='Premium'), 'A201', 2),
94 ((SELECT type_id FROM room_types WHERE type_name='Family'), 'A202', 2),
95 ((SELECT type_id FROM room_types WHERE type_name='Single-Deluxe'), 'B101', 1),
96 ((SELECT type_id FROM room_types WHERE type_name='Double-Deluxe'), 'B102', 1),
97 ((SELECT type_id FROM room_types WHERE type_name='Premium-Plus'), 'B201', 2),
98 ((SELECT type_id FROM room_types WHERE type_name='Family-Suite'), 'C301', 3),
99 ((SELECT type_id FROM room_types WHERE type_name='Economy'), 'C302', 3),
100 ((SELECT type_id FROM room_types WHERE type_name='Executive'), 'C303', 3);
```

77 13:43:21 INSERT INTO rooms (type_id, room_no, floor_no) VALUES ((SELECT type_id FROM room_types WHERE... 10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0

3. Insert at least 10 students

```
104 • INSERT INTO students (room_id, fname, lname, status, checkin_date, email) VALUES
105 (1, 'Ahmad', 'Zaki', 'ACTIVE', '2025-10-01', 'ahmad_z@graduate.utm.my'),
106 (2, 'Aisha', 'Binti', 'ACTIVE', '2025-09-15', 'aisha_b@graduate.utm.my'),
107 (3, 'Brian', 'Lim', 'ACTIVE', '2025-08-20', 'brian_lim@graduate.utm.my'),
108 (4, 'Cecilia', 'Tan', 'NON_ACTIVE', '2024-12-01', 'cecilia_t@graduate.utm.my'),
109 (5, 'Daniel', 'Lee', 'ACTIVE', '2025-10-10', 'daniel_l@graduate.utm.my'),
110 (6, 'Emily', 'Wong', 'NON_ACTIVE', '2023-05-05', 'emily_w@graduate.utm.my'),
111 (7, 'Farid', 'Hassan', 'ACTIVE', '2025-07-02', 'farid_h@graduate.utm.my'),
112 (8, 'Gina', 'Ong', 'ACTIVE', '2025-09-01', 'gina_o@graduate.utm.my'),
113 (9, 'Hafiz', 'Ibrahim', 'NON_ACTIVE', '2022-11-20', 'hafiz_i@graduate.utm.my'),
114 (10, 'Irene', 'Ng', 'ACTIVE', '2025-10-05', 'irene_ng@graduate.utm.my');
```

78 13:43:22 INSERT INTO students (room_id, fname, lname, status, checkin_date, email) VALUES (1, 'Ahmad', 'Zaki'... 10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0

4. Insert at least 10 maintenance records

```
118 • INSERT INTO maintenance (room_id, issue_desc, severity, status, reported_on, resolved_on) VALUES
119 (1, 'Water leakage in bathroom', 'MEDIUM', 'OPEN', DATE_SUB(CURDATE(), INTERVAL 10 DAY), NULL),
120 (2, 'Broken wardrobe hinge', 'LOW', 'RESOLVED', DATE_SUB(CURDATE(), INTERVAL 90 DAY), DATE_SUB(CURDATE(), INTERVAL 85 DAY)),
121 (3, 'Aircon not cooling', 'HIGH', 'OPEN', DATE_SUB(CURDATE(), INTERVAL 3 DAY), NULL),
122 (4, 'Cracked window pane', 'MEDIUM', 'RESOLVED', DATE_SUB(CURDATE(), INTERVAL 120 DAY), DATE_SUB(CURDATE(), INTERVAL 110 DAY)),
123 (5, 'Mold on ceiling', 'HIGH', 'OPEN', DATE_SUB(CURDATE(), INTERVAL 20 DAY), NULL),
124 (6, 'Light flickering', 'LOW', 'OPEN', DATE_SUB(CURDATE(), INTERVAL 5 DAY), NULL),
125 (7, 'Broken chair', 'LOW', 'RESOLVED', DATE_SUB(CURDATE(), INTERVAL 200 DAY), DATE_SUB(CURDATE(), INTERVAL 195 DAY)),
126 (8, 'Clogged drain', 'MEDIUM', 'OPEN', DATE_SUB(CURDATE(), INTERVAL 45 DAY), NULL),
127 (9, 'Door lock jammed', 'HIGH', 'OPEN', DATE_SUB(CURDATE(), INTERVAL 2 DAY), NULL),
128 (10, 'Peeling paint', 'LOW', 'RESOLVED', DATE_SUB(CURDATE(), INTERVAL 75 DAY), DATE_SUB(CURDATE(), INTERVAL 70 DAY));
```

79 13:43:22 INSERT INTO maintenance (room_id, issue_desc, severity, status, reported_on, resolved_on) VALUES (... 10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0

5. Insert at least 10 payments

```
132 • INSERT INTO payments (student_id, amount, paid_on, method, note) VALUES
133 (1, 350.00, '2025-09-01', 'CASH', 'September Rent'),
134 (2, 500.00, '2025-09-02', 'FPX', 'September Rent'),
135 (3, 900.00, '2025-09-05', 'CARD', 'September Rent'),
136 (5, 450.00, '2025-10-01', 'TNG', 'October Rent'),
137 (7, 600.00, '2025-10-03', 'FPX', 'October Rent'),
138 (8, 380.00, '2025-10-05', 'CARD', 'October Rent'),
139 (10, 350.00, '2025-09-15', 'CASH', 'September Rent late'),
140 (1, 350.00, '2025-10-01', 'FPX', 'October Rent'),
141 (2, 500.00, '2025-10-02', 'CARD', 'October Rent'),
142 (3, 900.00, '2025-10-06', 'CASH', 'October Rent');
```

80 13:43:22 INSERT INTO payments (student_id, amount, paid_on, method, note) VALUES (1, 350.00, '2025-09-01', ... 10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0

6. Update operations

```

145 -- Temporarily disable safe updates to allow full-table update
146 • SET SQL_SAFE_UPDATES = 0;
147
148 -- Step 1: Set all rooms to not occupied (FALSE)
149 • UPDATE rooms
150   SET is_occupied = FALSE;
151
152 -- Step 2: Set rooms to occupied (TRUE) if any ACTIVE student exists
153 • UPDATE rooms
154   SET is_occupied = TRUE
155   WHERE room_id IN (
156     SELECT DISTINCT room_id
157     FROM students
158     WHERE status = 'ACTIVE'
159   );
160
161 -- Re-enable safe update mode after update
162 • SET SQL_SAFE_UPDATES = 1;

```

209	14:06:22	SET SQL_SAFE_UPDATES = 0	0 row(s) affected
210	14:06:22	UPDATE rooms SET is_occupied = FALSE	0 row(s) affected Rows matched: 10 Changed: 0 Warnings: 0
211	14:06:22	UPDATE rooms SET is_occupied = TRUE WHERE room_id IN (SELECT DISTINCT room_id FROM students WHERE status = 'ACTIVE');	7 row(s) affected Rows matched: 7 Changed: 7 Warnings: 0
212	14:06:22	SET SQL_SAFE_UPDATES = 1	0 row(s) affected

7. Delete operations

```

161 -- Delete maintenance records where status = 'RESOLVED' and reported_on older than 60 days
162 • DELETE FROM maintenance
163   WHERE status = 'RESOLVED'
164   AND reported_on < (CURDATE() - INTERVAL 60 DAY);

```

255 14:12:00 DELETE FROM maintenance WHERE status = 'RESOLVED' AND reported_on < (CURDATE() - INTERVAL 60 DAY); 4 row(s) affected

8. Display room types where rent is between RM400 and RM800

```
171 • SELECT * FROM room_types WHERE rent BETWEEN 400.00 AND 800.00;
```

type_id	type_name	rent	deposit	capacity
2	Double	500.00	300.00	2
5	Single-Deluxe	450.00	250.00	1
6	Double-Deluxe	600.00	350.00	2
NULL	NULL	NULL	NULL	NULL

9. Retrieve students whose names (fname) start with the letter 'A'

```
174 • SELECT * FROM students WHERE fname LIKE 'A%';
```

student_id	room_id	fname	lname	status	checkin_date	email
1	1	Ahmad	Zaki	ACTIVE	2025-10-01	ahmad_z@graduate.utm.my
2	2	Aisha	Binti	ACTIVE	2025-09-15	aisha_b@graduate.utm.my
NULL	NULL	NULL	NULL	NULL	NULL	NULL

10. Display payments where the payment method is IN ('FPX', 'CARD')

```
177 • SELECT * FROM payments WHERE method IN ('FPX', 'CARD');
```

payment_id	student_id	amount	paid_on	method	note
2	2	500.00	2025-09-02	FPX	September Rent
3	3	900.00	2025-09-05	CARD	September Rent
5	7	600.00	2025-10-03	FPX	October Rent
6	8	380.00	2025-10-05	CARD	October Rent
8	1	350.00	2025-10-01	FPX	October Rent
9	2	500.00	2025-10-02	CARD	October Rent
NULL	NULL	NULL	NULL	NULL	NULL

11. Find all **ACTIVE** students who are **NOT in floor 2**, and either **made a payment above RM500** or have a **room with HIGH severity issue**

```

182 • SELECT s.student_id, s.fname, s.lname, s.status, r.room_no, m.severity, p.amount
183 FROM students s
184 JOIN rooms r ON s.room_id = r.room_id
185 LEFT JOIN payments p ON p.student_id = s.student_id
186 LEFT JOIN maintenance m ON m.room_id = r.room_id
187 WHERE s.status = 'ACTIVE'
188 AND NOT (r.floor_no = 2)
189 AND (p.amount > 500 OR m.severity = 'HIGH');

```

student_id	fname	lname	status	room_no	severity	amount
5	Daniel	Lee	ACTIVE	B101	HIGH	450.00

12. Aggregate function: Total number of active students

```

191 -- Aggregate function example: total number of active students
192 • SELECT COUNT(*) AS total_active_students FROM students WHERE status = 'ACTIVE';

```

total_active_students
7

13. String function

```

195 • SELECT student_id, UPPER(fname) AS fname_upper, UPPER(lname) AS lname_upper,
196        CONCAT(fname, ' ', lname) AS full_name
197 FROM students
198 LIMIT 10;

```

student_id	fname_upper	lname_upper	full_name
1	AHMAD	ZAKI	Ahmad Zaki
2	AISHA	BINTI	Aisha Binti
3	BRIAN	LIM	Brian Lim
4	CECILIA	TAN	Cecilia Tan
5	DANIEL	LEE	Daniel Lee
6	EMILY	WONG	Emily Wong
7	FARID	HASSAN	Farid Hassan
8	GINA	ONG	Gina Ong
9	HAFIZ	IBRAHIM	Hafiz Ibrahim
10	IRENE	NG	Irene Ng

Task 3: Reporting and Aggregation

1. Create a view v_room_status

```
200      -- Create a view v_room_status
201 •    DROP VIEW IF EXISTS v_room_status;
202
203 •    CREATE VIEW v_room_status AS
204      SELECT
205          r.room_no,
206          rt.type_name,
207          rt.rent,
208          r.floor_no,
209          rt.capacity,
210          COUNT(DISTINCT s.student_id) AS n_occupants,
211          COUNT(DISTINCT m.maint_id) AS pending_issues,
212          (COUNT(DISTINCT s.student_id) = 0) AS is_vacant
213      FROM rooms r
214      JOIN room_types rt ON r.type_id = rt.type_id
215      LEFT JOIN students s
216          ON s.room_id = r.room_id AND s.status = 'ACTIVE'
217      LEFT JOIN maintenance m
218          ON m.room_id = r.room_id AND m.status = 'OPEN'
219      GROUP BY r.room_id, r.room_no, rt.type_name, rt.rent, r.floor_no, rt.capacity;
220
221      -- To view the results of the view:
222 •    SELECT * FROM v_room_status ORDER BY room_no;
```

room_no	type_name	rent	floor_no	capacity	n_occupants	pending_issues	is_vacant
A101	Single	350.00	1	1	1	1	0
A102	Double	500.00	1	2	1	0	0
A201	Premium	900.00	2	2	1	1	0
A202	Family	1200.00	2	4	0	0	1
B101	Single-Deluxe	450.00	1	1	1	1	0
B102	Double-Deluxe	600.00	1	2	0	1	1
B201	Premium-Plus	1000.00	2	2	1	0	0
C301	Family-Suite	1500.00	3	5	1	1	0
C302	Economy	380.00	3	1	0	1	1
C303	Executive	2000.00	3	3	1	0	0

2. Total number of students per room type

```
224      -- Total number of students per room type
225 •    SELECT rt.type_name,
226          COUNT(s.student_id) AS total_students
227      FROM room_types rt
228      JOIN rooms r ON r.type_id = rt.type_id
229      LEFT JOIN students s ON s.room_id = r.room_id
230      GROUP BY rt.type_id, rt.type_name
231      ORDER BY total_students DESC;
```

type_name	total_students
Double	1
Double-Deluxe	1
Economy	1
Executive	1
Family	1
Family-Suite	1
Premium	1
Premium-Plus	1
Single	1
Single-Deluxe	1

3. Average rent and total deposit per room type (use ROUND() to 2 d.p.)

```
233 -- Average rent and total deposit per room type (use ROUND() to 2 d.p.)
234 • SELECT rt.type_name,
235         ROUND(AVG(rt.rent),2) AS avg_rent,
236         ROUND(SUM(rt.deposit),2) AS total_deposit
237 FROM room_types rt
238 GROUP BY rt.type_id, rt.type_name;
```

type_name	avg_rent	total_deposit
Single	350.00	200.00
Double	500.00	300.00
Premium	900.00	500.00
Family	1200.00	700.00
Single-Deluxe	450.00	250.00
Double-Deluxe	600.00	350.00
Premium-Plus	1000.00	600.00
Family-Suite	1500.00	800.00
Economy	380.00	150.00
Executive	2000.00	1000.00

4. Monthly payment totals grouped by year & month (ensure your dataset spans ≥2 months).

```
240 -- Monthly payment totals grouped by year & month (ensure dataset spans >=2 months)
241 • SELECT YEAR(paid_on) AS yr, MONTH(paid_on) AS mth, ROUND(SUM(amount),2) AS total_paid
242 FROM payments
243 GROUP BY YEAR(paid_on), MONTH(paid_on)
244 ORDER BY yr, mth;
```

yr	mth	total_paid
2025	9	2100.00
2025	10	3180.00

5. Count of OPEN maintenance issues per floor using HAVING COUNT > 2.

```
246 -- Count of OPEN maintenance issues per floor using HAVING COUNT > 2
247 • SELECT r.floor_no, COUNT(m.maint_id) AS open_issues
248 FROM maintenance m
249 JOIN rooms r ON m.room_id = r.room_id
250 WHERE m.status = 'OPEN'
251 GROUP BY r.floor_no
252 HAVING COUNT(m.maint_id) > 2;
```

floor_no	open_issues
1	3

6. Show summary of each room type using CASE(LOW < 400, MEDIUM 400-700, HIGH > 700)

```
254 -- Show summary of each room type using CASE(LOW < 400, MEDIUM 400-700, HIGH > 700)
255 • SELECT
256     CONCAT('Type: ', UPPER(rt.type_name), ' | Rent: RM', ROUND(rt.rent,2)) AS room_summary,
257     CASE
258         WHEN rt.rent < 400 THEN 'LOW'
259         WHEN rt.rent BETWEEN 400 AND 700 THEN 'MEDIUM'
260         ELSE 'HIGH'
261     END AS rent_category
262 FROM room_types rt;
```

room_summary	rent_category
Type: SINGLE Rent: RM350.00	LOW
Type: DOUBLE Rent: RM500.00	MEDIUM
Type: PREMIUM Rent: RM900.00	HIGH
Type: FAMILY Rent: RM1200.00	HIGH
Type: SINGLE-DELUXE Rent: RM450.00	MEDIUM
Type: DOUBLE-DELUXE Rent: RM600.00	MEDIUM
Type: PREMIUM-PLUS Rent: RM1000.00	HIGH
Type: FAMILY-SUITE Rent: RM1500.00	HIGH
Type: ECONOMY Rent: RM380.00	LOW
Type: EXECUTIVE Rent: RM2000.00	HIGH